



Made for humans,
powered by AI

Inteligência Artificial

Treinamento



Vitória Firmino
Product Owner

- Parte 1: entender como LLMs funcionam e quais são seus limites reais.
- Parte 2: decidir como usar LLMs em sistemas reais, com qualidade, custo, segurança e responsabilidade.



Arquiteturas



Tools

LLM puro é o modelo pré-treinado, com seus pesos aprendidos durante o treinamento, operando apenas sobre o contexto enviado no prompt.

Ele **não executa lógica externa**, apenas raciocina em linguagem.

Quando o produto precisa:

- Acessar dados atualizados
- Consultar informação privada
- Executar ação no sistema
- Tomar decisão com base em dado externo
- Integrar com APIs

É aqui que entram as **ferramentas (tools)**.

O que são Ferramentas (Tools / Function Calling)?

Ferramentas são **funções externas** que o modelo pode decidir chamar.

O LLM não executa a lógica.

Ele apenas decide qual ferramenta usar e com quais parâmetros.

Exemplos de Tools:

- API de Banco de Dados
- Busca Web
- Busca Vetorial (RAG)
- Ferramentas de Ação
- Ferramentas Determinísticas (Código, Calculadora, etc.)
- Ferramentas de Validação

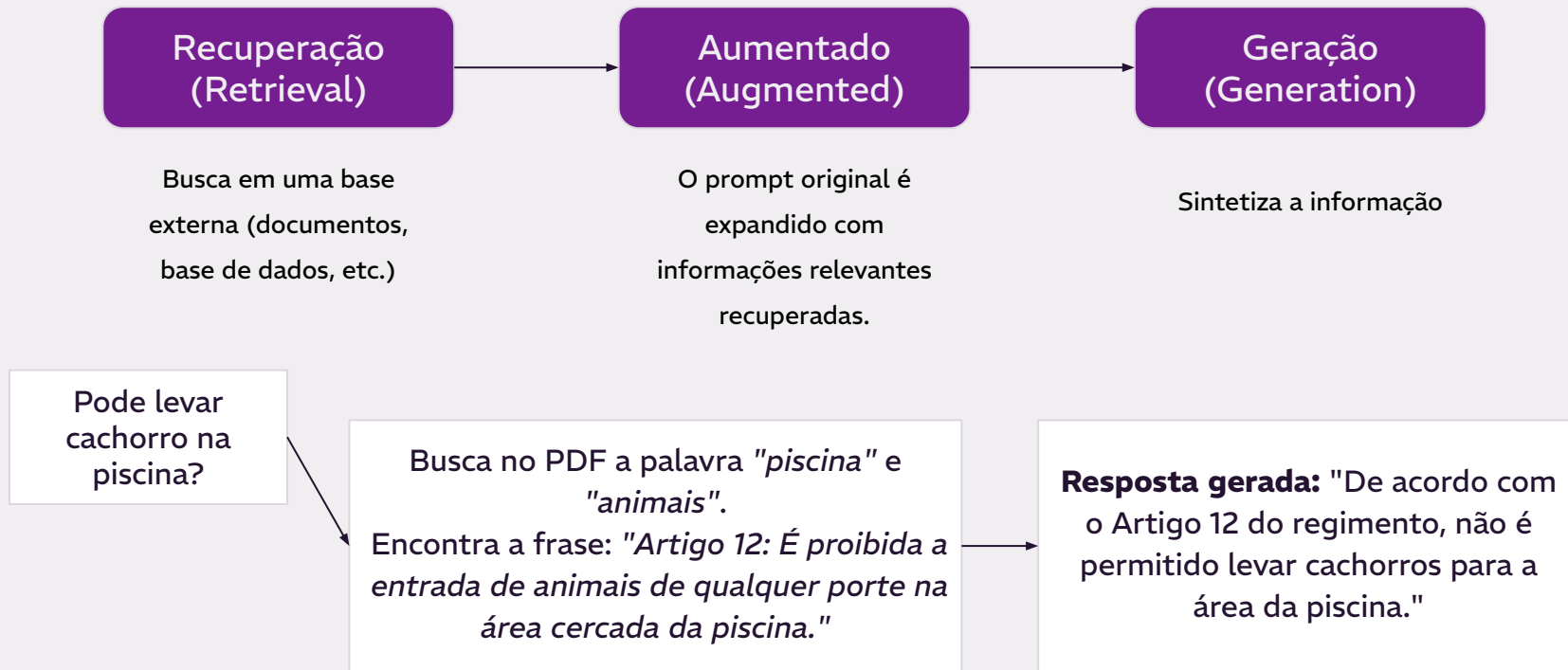
RAG (Retrieval-Augmented Generation)

RAG é uma arquitetura onde o modelo primeiro recupera informações relevantes de uma base externa e depois gera a resposta com base nelas.

RAG resolve principalmente três coisas:

1. **Factualidade:** Reduz alucinação porque o modelo responde com base em fonte específica.
2. **Conhecimento Proprietário:** Permite usar documentos internos, base jurídica, FAQ corporativo, Histórico de cliente, etc.
3. **Atualização Frequente:** Você atualiza a base e não precisa retreinar modelo.

RAG (Retrieval-Augmented Generation)



RAG (Retrieval-Augmented Generation)

Adicionar RAG implica:

- Pipeline de ingestão
- Chunking de documentos
- Embeddings
- Busca vetorial
- Ranking de relevância
- Monitoramento de qualidade de recuperação

Isso aumenta:

- Complexidade técnica
- Latência
- Custo por requisição
- Necessidade de observabilidade

Chunking e Busca Vetorial

Chunking é o processo de quebrar documentos grandes em pedaços menores (chunks) antes de indexar no banco vetorial.

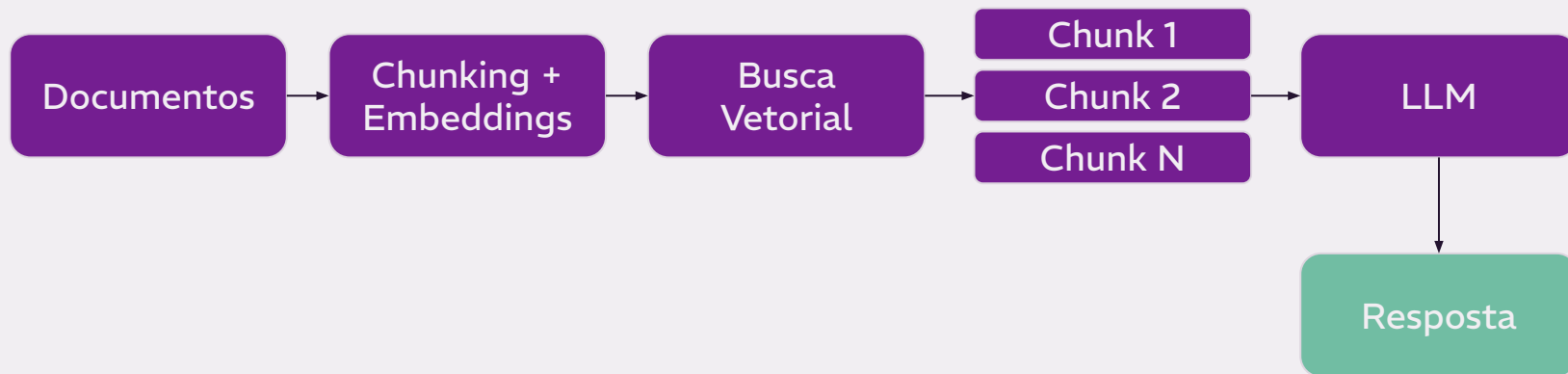
Porque um LLM não consegue ler 300 páginas de uma vez ou receber um PDF inteiro no contexto. Então dividimos o conteúdo em blocos menores.

Busca vetorial é o mecanismo que encontra os trechos mais semanticamente similares à pergunta do usuário.

1. Cada chunk vira um vetor numérico (embedding)
2. A pergunta do usuário também vira vetor
3. O sistema mede “proximidade semântica”
4. Retorna os mais parecidos

Depois que a busca vetorial encontra vários trechos candidatos, o sistema os ordena por relevância e envia os mais bem classificados para o LLM.

Pipeline RAG



Como avaliar?

RAGAS é um conjunto de métricas criado especificamente para avaliar sistemas com recuperação de informação (RAG), ajudando a identificar onde o pipeline está falhando.

1. Avalia se o sistema encontrou os trechos corretos.

- Context Precision: os trechos recuperados eram realmente relevantes?
- Context Recall: o sistema recuperou tudo que era necessário?
- Context Entities Recall: as entidades importantes foram encontradas?
- Noise Sensitivity: o modelo se confunde com informação irrelevante?

2. Avalia se o modelo respondeu corretamente com base no contexto.

- Faithfulness: a resposta está fiel aos trechos recuperados?
- Response Relevancy: a resposta realmente atende à pergunta?
- Answer Correctness: a resposta está correta factualmente?
- Answer Similarity: a resposta é semanticamente próxima do esperado?

Como avaliar?

RAGAS é um conjunto de métricas criado especificamente para avaliar sistemas com recuperação de informação (RAG), ajudando a identificar onde o pipeline está falhando.

1. Avalia se o sistema encontrou os trechos corretos.

- Context Precision: os trechos recuperados eram realmente relevantes?
- Context Recall: o sistema recuperou tudo que era necessário?
- Context Entities Recall: as entidades importantes foram encontradas?
- Noise Sensitivity: o modelo se confunde com informação irrelevante?

2. Avalia se o modelo respondeu corretamente com base no contexto.

- Faithfulness: a resposta está fiel aos trechos recuperados?
- Response Relevancy: a resposta realmente atende à pergunta?
- Answer Correctness: a resposta está correta factualmente?
- Answer Similarity: a resposta é semanticamente próxima do esperado?

Fine-tuning

Fine-tuning é o processo de ajustar um modelo pré-treinado com exemplos específicos para modificar seu comportamento.

Ele aprende padrões como:

- Tom específico
- Estrutura fixa
- Estilo padronizado
- Classificação especializada
- Formato rígido de saída

O que muda em custo e operação?

Fine-tuning implica:

- Dataset curado e de qualidade
- Processo de treinamento
- Validação
- Versionamento de modelo
- Monitoramento pós-deploy
- Custo inicial alto
- Tempo de implementação maior
- Dependência do fornecedor/modelo

Use Fine-tuning quando:

- O padrão de resposta é repetitivo
- O volume de uso é alto (justifica investimento)
- Prompt engineering já foi levado ao limite
- RAG não resolve o problema

Evite quando:

- Está ainda validando problema
- Volume é baixo
- Mudanças são frequentes
- O problema é falta de contexto

Aspecto	LLM Puro	RAG	Fine-Tuning	Agentes AI
Resolve principalmente	Tarefas linguísticas gerais	Falta de conhecimento contextual ou atualizado	Inconsistência de estilo, formato ou comportamento	Problemas complexos que exigem decisão e ação
Não resolve	Dados atualizados ou proprietários	Comportamento desalinhado	Informação dinâmica	Necessidade de dados confiáveis sem RAG
Acesso a dados externos	Não	Sim	Não (diretamente)	Sim (via tools)
Execução de ações	Não	Não	Não	Sim
Complexidade técnica	Baixa	Média	Média/Alta	Alta
Custo operacional	Baixo	Médio	Alto (inicial)	Alto
Latência	Baixa	Média	Baixa	Alta
Tempo de implementação	Rápido	Médio	Longo	Longo
Melhor para	Resumo, classificação, geração simples	Chatbot com base interna, jurídico, helpdesk	Padronização em larga escala	Assistentes autônomos, copilotos operacionais

Avaliação de Qualidade em Modelos Generativos



Por que métricas tradicionais não bastam?

Métricas clássicas funcionam quando o output é um rótulo. LLMs geram texto aberto, múltiplas respostas possíveis e conteúdo probabilístico.

Modelos generativos exigem avaliação multidimensional.

Avaliação Humana vs Automática

Avaliação Humana

- Contextual
- Capaz de perceber nuance
- Avalia qualidade subjetiva

Porém:

- Cara
- Demorada
- Difícil de escalar

Ideal para:

- Features críticas
- Ajuste fino de comportamento

Avaliação Automática

- Escalável
- Rápida
- Reprodutível

Porém:

- Pode não capturar nuance
- Depende de métrica bem definida

Ideal para:

- Monitoramento contínuo
- Testes em larga escala
- A/B testing

Avaliação Critérios Comuns de Avaliação

Em sistemas generativos, qualidade é composta por múltiplos critérios:

Relevância: A resposta responde à pergunta?

Completeness: Cobre os pontos importantes?

Factualidade: Está baseada em fatos corretos?

Clareza: É compreensível, organizada e objetiva?

Qualidade precisa ser definida antes de ser medida.

LLM-as-a-Judge

Usar um LLM para avaliar a saída de outro LLM.

Exemplo: **Modelo A** gera resposta e **Modelo B** avalia segundo critérios.

Ele pode medir: Relevância, Factualidade, Coerência, Comparação entre respostas, etc.

Como funciona?

- Define critérios claros
- Força avaliação estruturada
- Recebe score ou justificativa

A/B Testing para Features com LLM

Em produtos com LLM, podemos testar:

- Prompt A vs Prompt B
- Modelo A vs Modelo B
- RAG vs sem RAG
- Temperatura diferente
- Pipeline simples vs multi-etapas

Combine:

- Métricas comportamentais
- Métricas de qualidade
- Feedback qualitativo

Observabilidade e Logging

Em software tradicional:

- Se algo quebra → erro 500
- Stack trace
- Log técnico

Em LLM:

- A resposta pode estar errada
- Mas não existe "erro técnico"
- Não há exceção
- Não há crash

O sistema continua funcionando mas com qualidade degradada.

O que é Observabilidade?

Observabilidade é a capacidade de entender:

- O que o modelo recebeu
- O que ele respondeu
- Com base em qual contexto
- Usando qual versão
- Chamando quais ferramentas
- Com quais parâmetros

Sem isso, você não consegue:

- Depurar
- Auditar
- Melhorar
- Defender decisões

O que deve ser logado?

Sistemas com LLM precisam registrar, no mínimo:

- Prompt completo enviado ao modelo (incluindo system prompt)
- Resposta gerada
- Contexto adicional (ex: chunks de RAG)
- Versão do modelo (ex: gpt-4.1, fine-tuned v3)
- Parâmetros de geração (temperatura, top-p)
- Chamadas de ferramentas (nome da função + argumentos)
- Resultado da ferramenta
- Score de avaliação automática

Observabilidade em RAG

Em sistemas com RAG, você deve monitorar:

- Quais chunks foram recuperados?
- Quantos?
- Em que ordem?
- Qual score de similaridade?
- Houve ruído?
- O modelo citou as fontes corretamente?

Se a resposta estiver errada, você precisa saber:

Foi erro de busca?

Ou erro de geração?

Observabilidade em Agentes

Para agentes, é ainda mais crítico. Você deve logar:

- Plano gerado
- Decisões intermediárias
- Chamadas de tools
- Falhas de tool
- Re-tentativas
- Estados intermediários

Custos e Escalabilidade



Custo

Diferente de software tradicional:

- O custo não é fixo
- O custo varia por uso
- O custo depende de arquitetura
- O custo depende do tamanho do contexto

LLM é custo por token.

Em geral, o custo depende de:

- Tokens de entrada (prompt + contexto)
- Tokens de saída (resposta gerada)
- Número de chamadas
- Modelo escolhido

Quanto maior o contexto, maior o custo. Quanto mais etapas no pipeline, maior o custo.

Trade-off

Mais contexto significa mais informação e quando bem utilizado maior performance. Porém, quanto maior a quantidade de tokens, maior o custo e latência.

Precisamos sempre nos perguntar:

- Eu preciso mesmo enviar todo esse histórico?
- Eu preciso do modelo mais caro para essa tarefa?
- Esse ganho de qualidade compensa o aumento de custo?

Reduzir contexto ou utilizar um modelo mais leve pode reduzir significativamente o custo sem comprometer o valor entregue.

Impacto Financeiro de Erro

Erro em LLM pode gerar:

- Retrabalho humano
- Perda de confiança
- Risco jurídico
- Custo reputacional

Às vezes, pagar mais por qualidade sai mais barato.

Escalabilidade

- A margem de lucro por usuário se mantém à medida que o uso aumenta? Se o custo de tokens cresce na mesma proporção que a receita, o modelo de negócio pode ser insustentável.
- Estamos usando "um canhão para matar uma mosca"? Posso mover tarefas simples para modelos menores sem perda de qualidade perceptível?
- Qual é o teto de gastos por usuário/empresa? Como evitar que um único usuário consuma todo o orçamento de API do mês em um dia?
- O tempo de resposta está dentro do aceitável?
- Como o sistema se comporta sob carga simultânea? Se 1.000 usuários perguntarem ao mesmo tempo, a latência degrada exponencialmente ou linearmente?
- Existe um plano de contingência (fallback)? Se o provedor do modelo (OpenAI) cair?
- Como sabemos, em tempo real, se o modelo começou a alucinar mais após uma mudança? Temos métricas rodando em produção?
- Quão difícil é trocar o modelo base hoje? Estamos presos a um único provedor ou nossa arquitetura é agnóstica?
- Temos um "Golden Dataset" para testes de regressão? Sempre que atualizamos o prompt ou o modelo, como garantimos que o que funcionava ontem continua funcionando hoje?

Guardrails, Confiabilidade e Segurança



O que são Guardrails?

Guardrails são mecanismos arquiteturais que:

- Limitam o comportamento do modelo
- Controlam saída
- Reduzem risco
- Garantem conformidade
- Impedem ações indevidas

Eles não deixam o modelo “livre”.

Guardrails de Entrada

Filtro de Conteúdo Sensível

- Bloquear solicitações ilegais
- Detectar discurso de ódio
- Identificar tentativa de fraude

Exemplo:

Usuário pede instrução para hackear sistema → bloqueia antes de chegar ao modelo.

Detecção de Dados Pessoais

- CPF
- Número de cartão
- Dados médicos

Pode:

- Anonimizar
- Bloquear
- Alertar

Limite de Tamanho

- Restringir número de tokens
- Evitar upload excessivo

Protege custo e estabilidade.

Guardrails de Prompt

Papel e Limites Explícitos

Exemplo:

“Você não deve fornecer aconselhamento jurídico vinculante.”

Escopo Restrito

Exemplo:

“Responda apenas com base nos documentos fornecidos.”

Instrução de Incerteza

Exemplo:

“Se não houver informação suficiente, diga que não encontrou evidência.”

Guardrails de Saída

Saída Estruturada (JSON obrigatório)

Força formato fixo:

- Evita texto livre
- Facilita validação
- Permite bloquear campos vazios

Validação de Schema

Se faltar campo obrigatório:

- Rejeita resposta
- Solicita regeneração

Score de Confiança

Se confiança < limite:

- Não publica
- Solicita revisão humana

Classificação Pós-Geração

Rodar modelo secundário para:

- Detectar toxicidade
- Detectar risco jurídico
- Detectar afirmação categórica

Guardrails de RAG

Exigir Evidência

Modelo deve citar:

- Fonte
- Trecho
- Documento

Se não houver evidência →
bloquear.

Bloquear Resposta Fora do Contexto

Se pergunta não está na base:

- “Não encontrei informação suficiente.”

Guardrails de Agentes

Human-in-the-loop

Antes de:

- Enviar e-mail
- Aprovar transação
- Deletar dado

Exigir confirmação humana.

Limite de Ação

Agente pode:

- Sugerir envio

Mas não pode:

- Enviar automaticamente

Limite de Tentativas

Delimita número de tentativas.

Evita loop infinito.

Ética, Viés e Responsabilidade



LLMs não são neutros

Modelos de linguagem aprendem padrões estatísticos de grandes corpora.

Se os dados contêm:

- Estereótipos
- Assimetrias sociais
- Representações desiguais
- O modelo tende a reproduzi-los.

[Blodgett et al. \(2020\)](#) alertam que sistemas de NLP frequentemente refletem hierarquias sociais presentes nos dados e que viés não é apenas erro estatístico, mas dano social.

Viés em LLMs

Em LLMs generativos, o problema é ampliado:

- O modelo pode gerar conteúdo ofensivo
- Pode reforçar estereótipos
- Pode alucinar características de grupos

[Bender et al. \(2021\)](#) alertam que modelos de larga escala amplificam padrões problemáticos ao “estocarem” linguagem da internet.

Transparência e Explicabilidade

Explicabilidade em LLMs é desafiadora porque:

- São modelos de caixa-preta
- Pequenas mudanças no prompt alteram saída
- Há não-determinismo

Em LLMs, isso implica:

- Logs detalhados
- Versionamento
- Evidência explícita (RAG)
- Declaração de limites

Fairness-by-Design

Fairness não pode ser auditoria posterior.

- Deve estar no ciclo de desenvolvimento:
- Definição clara de escopo
- Escolha de dados
- Critérios de avaliação
- Monitoramento contínuo

[Friedman & Nissenbaum \(1996\)](#) já argumentavam que **valores** são embutidos no **design tecnológico**.

Uso Responsável de Dados

Princípios centrais:

- Minimização
- Finalidade
- Transparência
- Responsabilização

LGPD exige:

- Base legal clara
- Direito à explicação
- Não discriminação
- Prestação de contas

Conclusão

LLMs ampliam capacidades humanas.

Mas também amplificam vieses estruturais.

A literatura mostra:

- Viés é inerente a dados históricos
- Escala aumenta impacto
- Avaliação contínua é indispensável
- Design determina responsabilidade

Conclusão



Conclusão

Durante esse curso, nós não aprendemos apenas como usar LLMs. Nós entendemos que eles não pensam, não sabem, não verificam fatos e não possuem intenção. São **máquinas probabilísticas** que operam sobre **padrões estatísticos** de linguagem. E, ainda assim, produzem algo que **se parece** muito com **inteligência**.

LLMs **não são mágicos** mas **são poderosos**. Eles ampliam nossa capacidade de síntese, de produção, de organização de informação e de tomada de decisão. Eles reduzem fricções cognitivas. Eles aceleram fluxos. Eles escalam linguagem.

Mas tudo que amplifica capacidade, amplifica também **risco**. E a arquitetura faz toda a diferença. Ao longo do curso, vimos que o **LLM não é o sistema**. Ele é apenas um **componente** dentro de um **pipeline** que precisa de dados bem tratados, validação, observabilidade, controle de custo, fallback, métricas e supervisão humana quando necessário.

A pergunta que fica não é “como usar IA?”. A pergunta é: estamos prontos para assumir a responsabilidade de projetar sistemas probabilísticos que influenciam decisões em escala?

Bibliografia

Russell, S., & Norvig, P. (2009). Artificial Intelligence: A Modern Approach (3rd ed.). Pearson.

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.

Flach, P. (2012). Machine Learning: The Art and Science of Algorithms that Make Sense of Data. Cambridge University Press.

Haykin, S. (2008). Neural Networks and Learning Machines (3rd ed.). Prentice Hall.

Anthony, M., & Bartlett, P. L. (2009). Neural Network Learning: Theoretical Foundations. Cambridge University Press.

Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.